Program:

```
#include <DHT.h>

#define DHTPIN 4
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

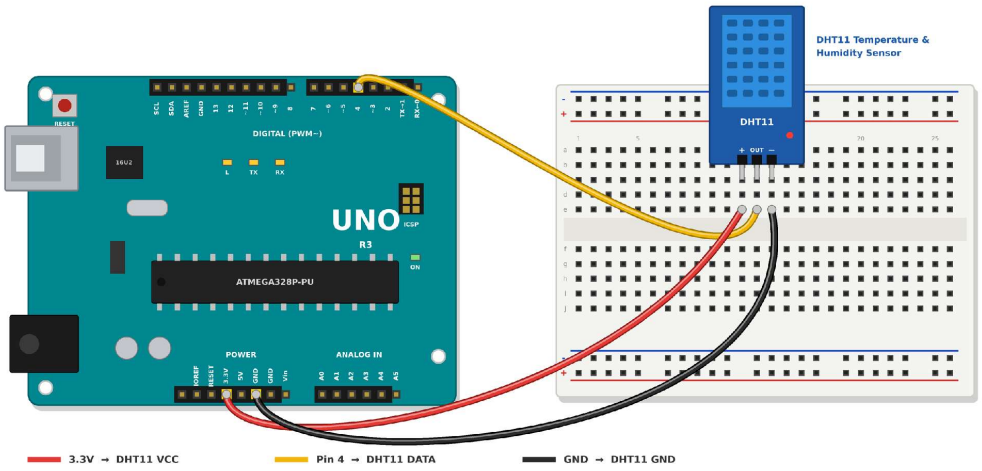
void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();

  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }

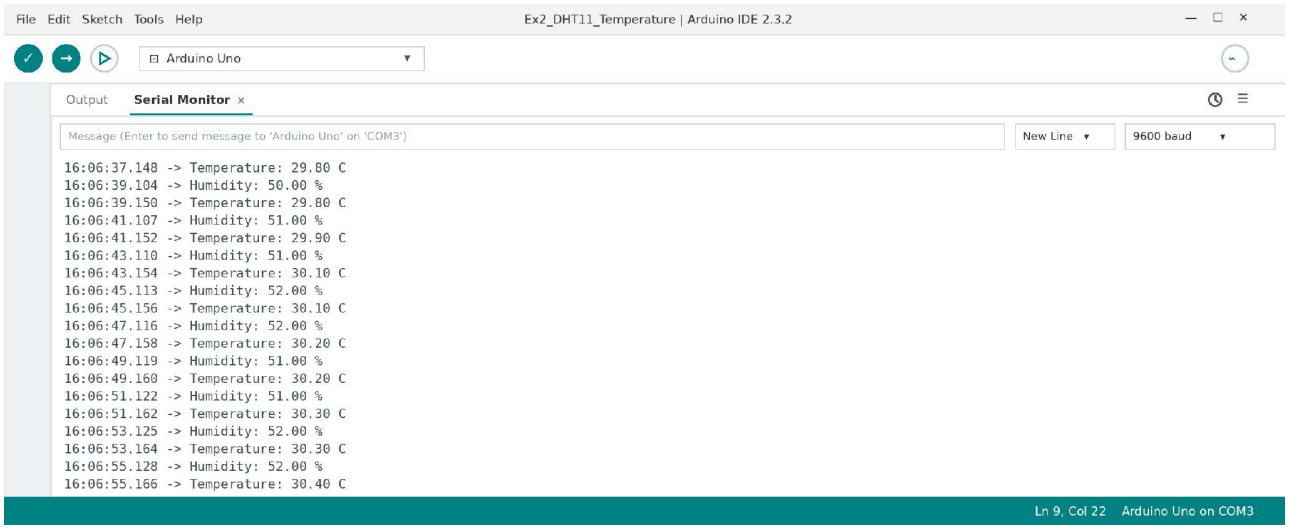
  Serial.print("Humidity: ");
  Serial.print(humidity);
  Serial.println(" %");
  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" C");
  delay(2000);
}
```

Circuit Diagram:



DHT11 Pin	Arduino UNO Pin
DATA (Output)	Digital Pin 4
VCC	3.3V
GND	GND

Output:



Serial Monitor output in Arduino IDE

ROOM TEMPERATURE MONITORING ON CLOUD PLATFORM

Program:

```
import network
import urequests
import time
import dht
from machine import Pin

# Wi-Fi and ThingSpeak details
SSID = "Your_WiFi_Name"
PASSWORD = "Your_WiFi_Password"
API_KEY = "YOUR_WRITE_API_KEY"
URL = "http://api.thingspeak.com/update"

sensor = dht.DHT11(Pin(15))

# Connect to Wi-Fi
wifi = network.WLAN(network.STA_IF)
wifi.active(True)
wifi.connect(SSID, PASSWORD)

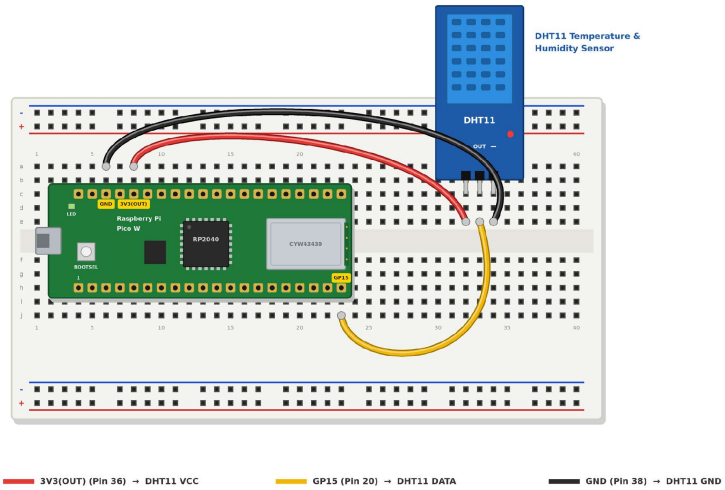
print("Connecting to Wi-Fi", end="")
while not wifi.isconnected():
    print(".", end="")
    time.sleep(1)
print("\nConnected to Wi-Fi")
print("IP Address:", wifi.ifconfig()[0])

while True:
    try:
        sensor.measure()
        temp = sensor.temperature()
        hum = sensor.humidity()
        print("Temperature:", temp, "C  Humidity:", hum, "%")

        response = urequests.get(URL + "?api_key=" + API_KEY +
                                 "&field1=" + str(temp) +
                                 "&field2=" + str(hum))
        print("Data sent to ThingSpeak, Entry ID:", response.text)
        response.close()
    except Exception as e:
        print("Error:", e)

    time.sleep(20)
```

Circuit Diagram:



DHT11 Pin	Pico W Pin Name	Physical Pin No.
VCC (+)	3V3(OUT)	36
DATA (OUT)	GP15	20
GND (−)	GND	38

Output:

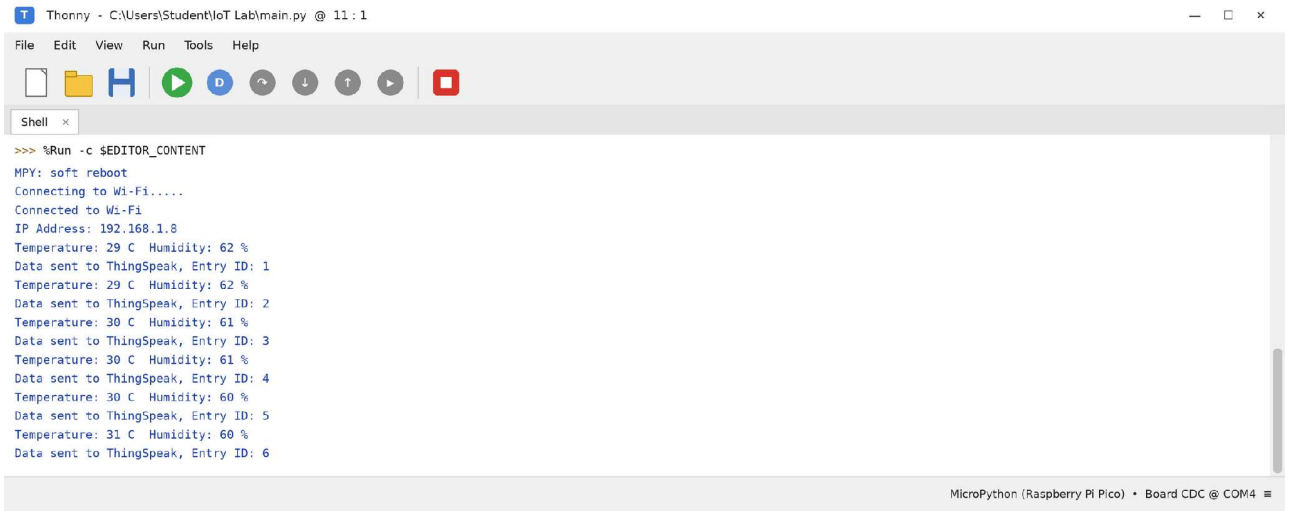


Figure 1: Output in Thonny Shell

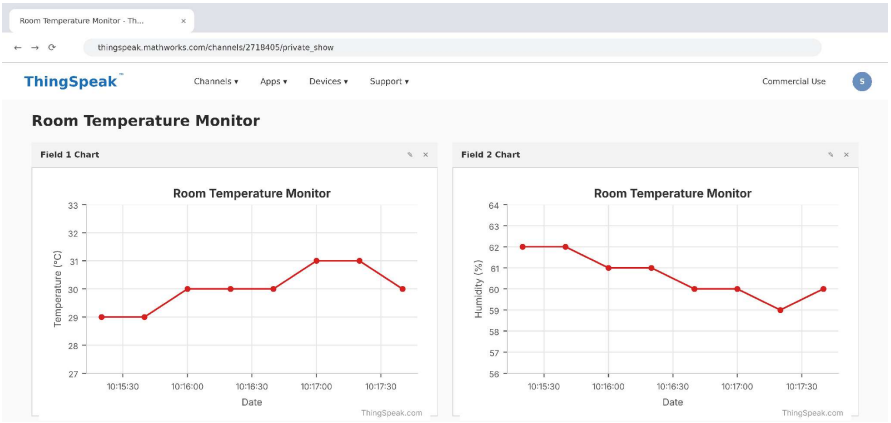


Figure 2: Values posted on ThingSpeak cloud

SOIL MOISTURE MONITORING USING GOOGLE FIREBASE

Program:

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClientSecure.h>

const char* ssid = "Your_WiFi_Name";
const char* password = "Your_WiFi_Password";
String firebaseURL = "https://soil-moisture-iot-default-rtdb.firebaseio.com/";

#define SOIL_PIN A0
#define LED_PIN D6

void setup() {
  pinMode(SOIL_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);

  WiFi.begin(ssid, password);
  Serial.print("Connecting to Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(500);
  }
  Serial.println();
  Serial.println("Connected to Wi-Fi");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
}

void loop() {
  int x = analogRead(SOIL_PIN);
  String status;

  if (x < 300) {
    digitalWrite(LED_PIN, HIGH);
    status = "Wet";
  } else {
    digitalWrite(LED_PIN, LOW);
    status = "Dry";
  }

  Serial.print("Soil Moisture Value: ");
  Serial.print(x);
  Serial.println("  (" + status + ")");

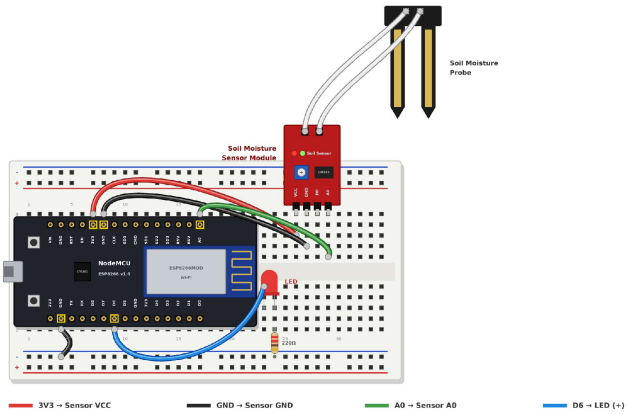
  // Send data to Firebase
  WiFiClientSecure client;
  client.setInsecure();
  HTTPClient http;
  http.begin(client, firebaseURL + "soil_moisture.json");
  http.addHeader("Content-Type", "application/json");

  String data = "{\"value\":" + String(x) + ",\"status\":\"" + status + "\"}";
  int code = http.POST(data);

  if (code == 200)
    Serial.println("Data sent to Firebase");
  else
    Serial.println("Failed to send data, Error code: " + String(code));
```

```
 http.end();
 delay(5000);
}
```

Circuit Diagram:



Component Pin	NodeMCU Pin
Soil Sensor VCC	3V3
Soil Sensor GND	GND
Soil Sensor A0	A0
LED (+)	D6
LED (−)	GND (through 220Ω)

Output:

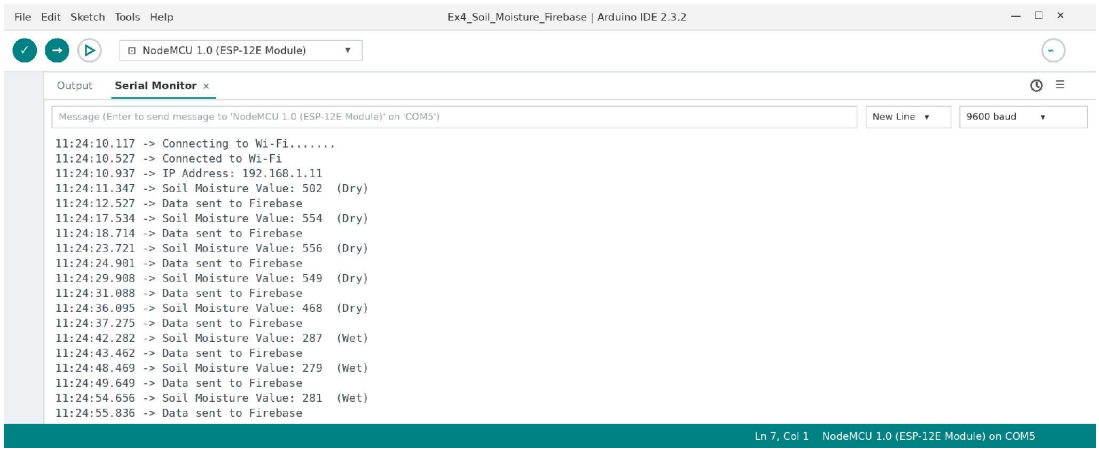


Figure 1: Soil moisture values in Serial Monitor

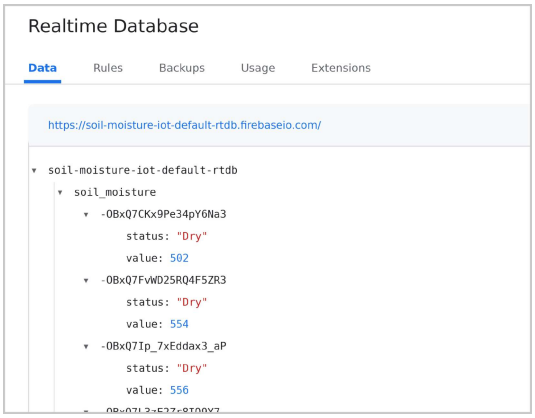


Figure 2: Data posted on Google Firebase

LED BLINKING APPLICATION USING RASPBERRY PI

Program:

Program 1 – Raspberry Pi (RPi.GPIO):

```
import RPi.GPIO as GPIO
import time

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BOARD)
GPIO.setup(11, GPIO.OUT, initial=GPIO.LOW)

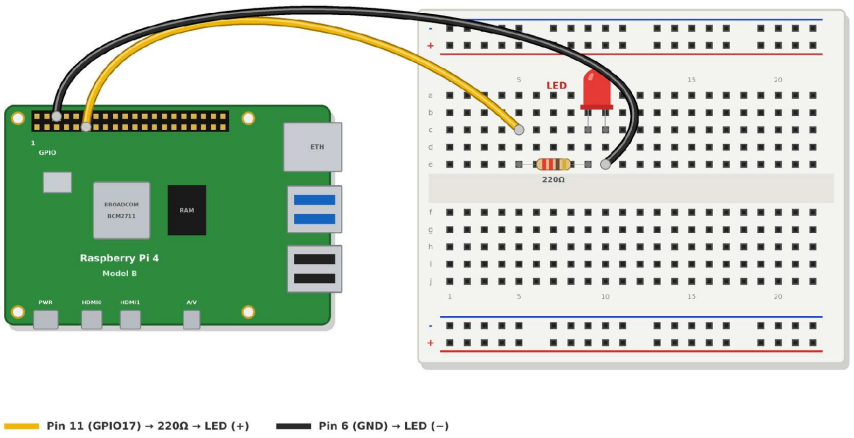
while True:
    GPIO.output(11, GPIO.HIGH)
    print("LED ON")
    time.sleep(1)
    GPIO.output(11, GPIO.LOW)
    print("LED OFF")
    time.sleep(1)
```

Program 2 – Raspberry Pi Pico (MicroPython):

```
from machine import Pin
import time

# GPIO 4
LED_PIN = 4
led = Pin(LED_PIN, Pin.OUT)
print("Blinking LED, Press Ctrl+C to stop")
try:
    while True:
        led.on()
        time.sleep(1)
        led.off()
        time.sleep(1)
except KeyboardInterrupt:
    print("Stopped by user")
```

Circuit Diagram:



Component	Raspberry Pi Pin (Program 1)	Raspberry Pi Pico Pin (Program 2)
LED (+) through 220Ω	Pin 11 (GPIO17)	GP4 (Pin 6)
LED (−)	Pin 6 (GND)	GND (Pin 3)

Output:

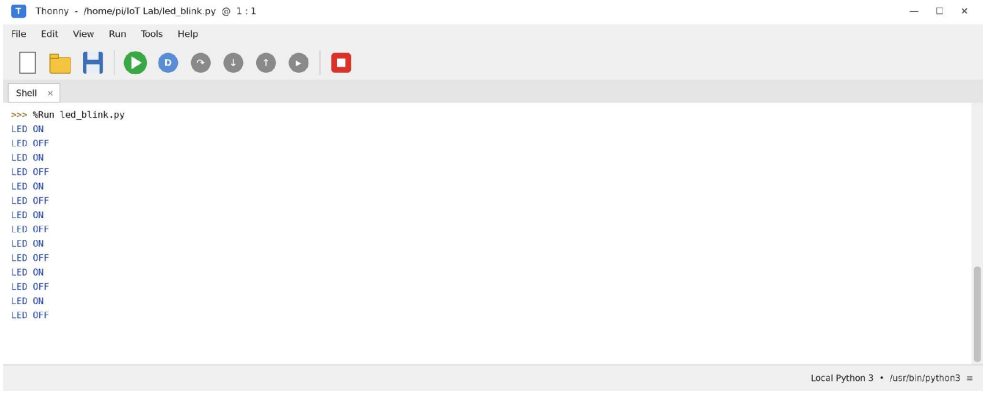


Figure 1: Output in Thonny Shell

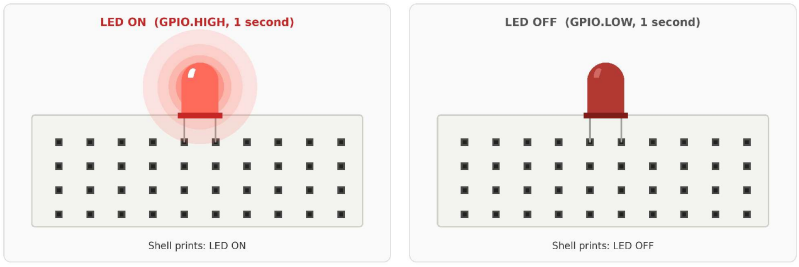


Figure 2: LED ON for 1 second and OFF for 1 second

SEND AND RECEIVE DATA USING HTTP REQUEST

Program:

```
#include <ESP8266WiFi.h>

const char* ssid = "Your_Network";
const char* password = "Network_Password";

int LED = 16;            // GPIO16 = D0 pin of NodeMCU
WiFiServer server(80);

void setup() {
  Serial.begin(115200);
  pinMode(LED, OUTPUT);
  digitalWrite(LED, LOW);

  Serial.print("Connecting to Internet");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");

  server.begin();
  Serial.println("Server started");
  Serial.print("IP Address of network: ");
  Serial.println(WiFi.localIP());
  Serial.print("Copy and paste the following URL: http://");
  Serial.print(WiFi.localIP());
  Serial.println("/");
}

void loop() {
  WiFiClient client = server.available();
  if (!client) {
    return;
  }

  Serial.println("Waiting for new client");
  while (!client.available()) {
    delay(1);
  }

  String request = client.readStringUntil('\r');
  Serial.println(request);
  client.flush();

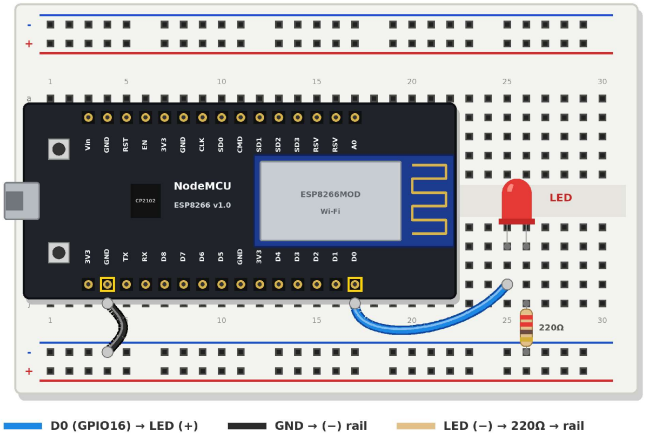
  int value = LOW;
  if (request.indexOf("/LED=ON") != -1) {
    digitalWrite(LED, HIGH);
    value = HIGH;
  }
  if (request.indexOf("/LED=OFF") != -1) {
    digitalWrite(LED, LOW);
    value = LOW;
  }

  client.println("HTTP/1.1 200 OK");
  client.println("Content-Type: text/html");
  client.println("");
```

```
  client.println("<!DOCTYPE HTML>");
  client.println("<html>");
  client.print("LED is ");
  if (value == HIGH) {
    client.print("ON");
  } else {
    client.print("OFF");
  }
  client.println("<br><br>");
  client.println("<a href=\"/LED=ON\"><button>ON</button></a>");
  client.println("<a href=\"/LED=OFF\"><button>OFF</button></a>");
  client.println("</html>");

  delay(1);
  Serial.println("Client disconnected");
  Serial.println("");
}
```

Circuit Diagram:



Component Pin	NodeMCU Pin
LED (+)	D0 (GPIO16)
LED (−)	GND (through 220Ω)

Output:



Figure 1: IP address and HTTP requests in Serial Monitor



Figure 2: Web page in browser after clicking ON

DISTANCE MEASUREMENT USING ULTRASONIC SENSOR AND CLOUD PLATFORM

Program:

```
from machine import Pin, Timer
import urequests as requests
import network
import time

firebase_url = "https://ultrasonic-iot-default-rtdb.firebaseio.com/"

TRIG = Pin(18, Pin.OUT)
ECHO = Pin(22, Pin.IN)

wifi = network.WLAN(network.STA_IF)
wifi.active(True)
wifi.connect("Your_WiFi_Name", "Your_WiFi_Password")

timeout = 30
start_time = time.time()
while not wifi.isconnected():
    if time.time() - start_time > timeout:
        print("Failed to connect Wi-Fi")
        break
    print("Connecting to Wi-Fi...")
    time.sleep(1)

if wifi.isconnected():
    print("Connected to Wi-Fi")
else:
    print("Not connected to Wi-Fi")

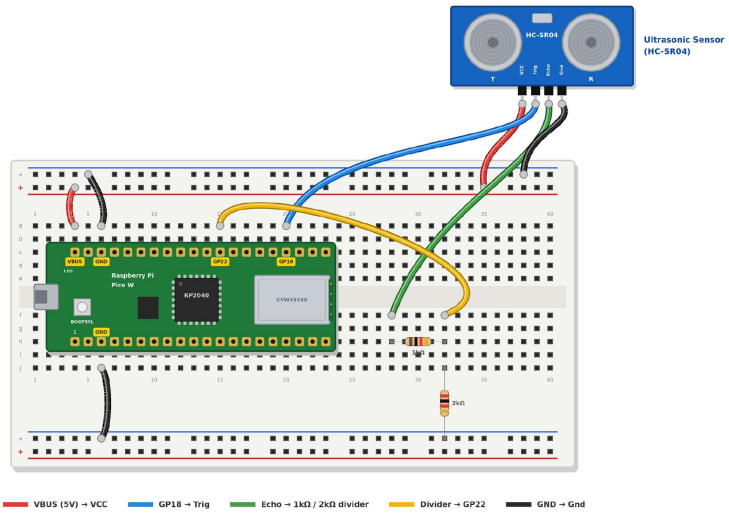
def read_distance():
    TRIG.value(0)
    time.sleep_us(2)
    TRIG.value(1)
    time.sleep_us(10)
    TRIG.value(0)
    pulse_start = time.ticks_us()
    while not ECHO.value():
        pulse_start = time.ticks_us()
    while ECHO.value():
        pulse_end = time.ticks_us()
    pulse_duration = time.ticks_diff(pulse_end, pulse_start)
    distance = pulse_duration * 0.0343 / 2
    distance = round(distance, 2)
    return distance

def send_data(timer):
    try:
        distance = read_distance()
        print("Distance:", distance, "cm")
        data = {'distance_cm': distance, 'timestamp': int(time.time())}
        response = requests.post(firebase_url + "distance.json", json=data)
        if response.status_code == 200:
            print("Distance data sent to Firebase")
        else:
            print("Failed to send data to Firebase:", response.text)
        response.close()
    except Exception as e:
        print("Error:", e)

tim = Timer()
```

```
tim.init(period=2000, mode=Timer.PERIODIC, callback=send_data)
```

Circuit Diagram:



HC-SR04 Pin	Pico W Pin Name	Physical Pin No.
VCC	VBUS (5V)	40
Trig	GP18	24
Echo (through divider)	GP22	29
Gnd	GND	38

Output:

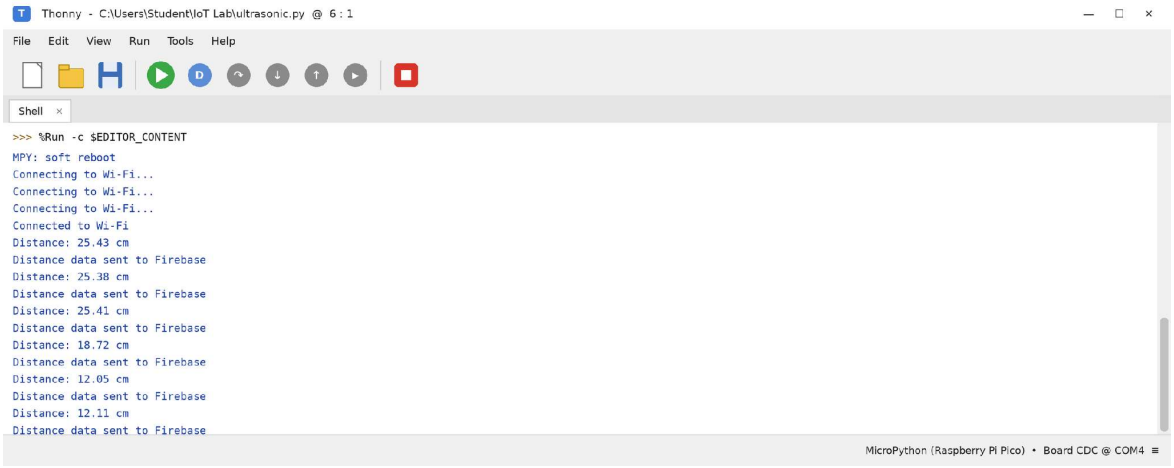


Figure 1: Distance values in Thonny Shell

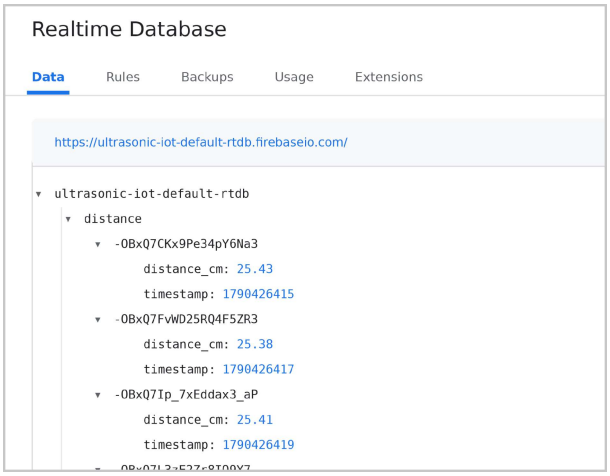


Figure 2: Distance values posted on Google Firebase

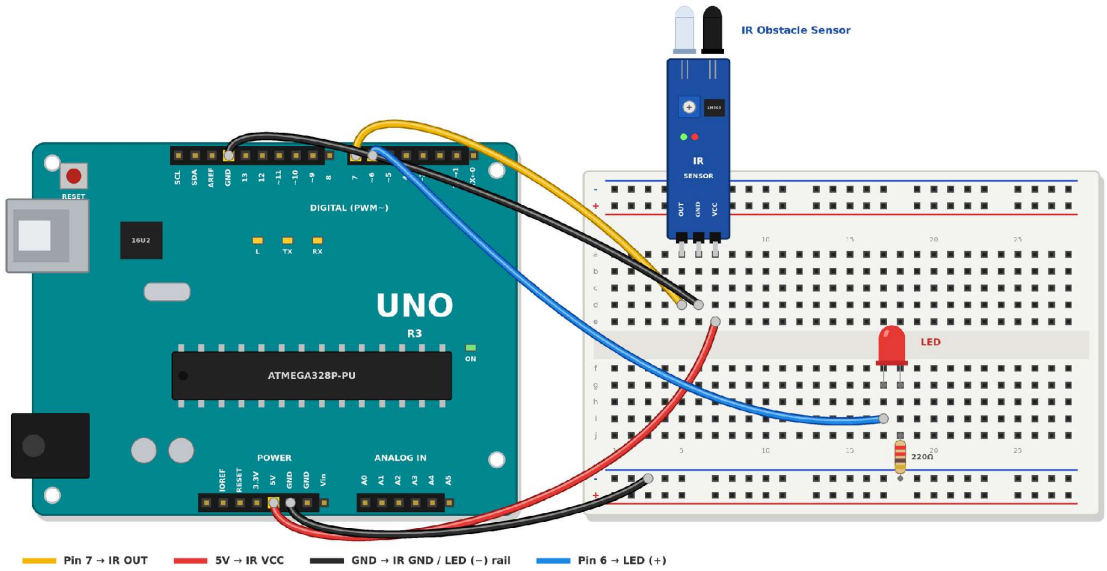
INFRARED SENSORS USING ARDUINO UNO

Program:

```
void setup() {
  pinMode(7, INPUT);
  pinMode(6, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int x = digitalRead(7);
  Serial.println(x);
  delay(1000);
  if (x == 0)
    digitalWrite(6, HIGH);
  else
    digitalWrite(6, LOW);
}
```

Circuit Diagram:



Component Pin	Arduino UNO Pin
IR Sensor OUT	Digital Pin 7
IR Sensor VCC	5V
IR Sensor GND	GND
LED (+) through 220Ω	Digital Pin 6
LED (−)	GND

Output:

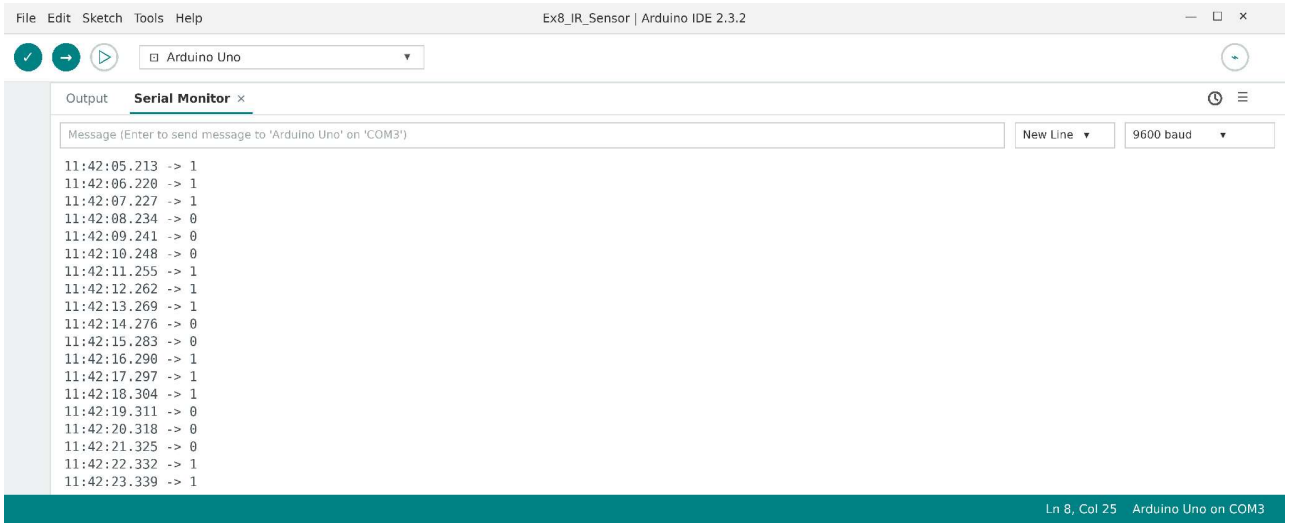


Figure: IR sensor values in Serial Monitor (1 = no object, 0 = object detected, LED ON)

Program:

fireapp/views.py:

```
from django.shortcuts import render
import pyrebase

config = {
    "apiKey": "YOUR_API_KEY",
    "authDomain": "django-iot-lab.firebaseapp.com",
    "databaseURL": "https://django-iot-lab-default-rtdb.firebaseio.com",
    "projectId": "django-iot-lab",
    "storageBucket": "django-iot-lab.appspot.com",
    "messagingSenderId": "YOUR_SENDER_ID",
    "appId": "YOUR_APP_ID",
    "measurementId": "YOUR_MEASUREMENT_ID"
}

firebase = pyrebase.initialize_app(config)
auth = firebase.auth()
database = firebase.database()

def index(request):
    Department = database.child('Data').child('Department').get().val()
    Name = database.child('Data').child('Name').get().val()
    RollNo = database.child('Data').child('RollNo').get().val()
    context = {
        'Department': Department,
        'Name': Name,
        'RollNo': RollNo,
    }
    return render(request, 'index.html', context)
```

firebase/urls.py:

```
from django.contrib import admin
from django.urls import path
from fireapp import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', views.index, name='index'),
]
```

fireapp/templates/index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Firebase App</title>
</head>
<body>
    <h1>Student Information</h1>
    <h3>* This is {{ Department }} Department</h3>
    <h3>* {{ Name }} is the student studying in {{ Department }}</h3>
    <h3>* His Roll No is {{ RollNo }}</h3>
</body>
</html>
```

firebase/settings.py (INSTALLED_APPS):

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
```

```
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'fireapp',
]
```

Output:

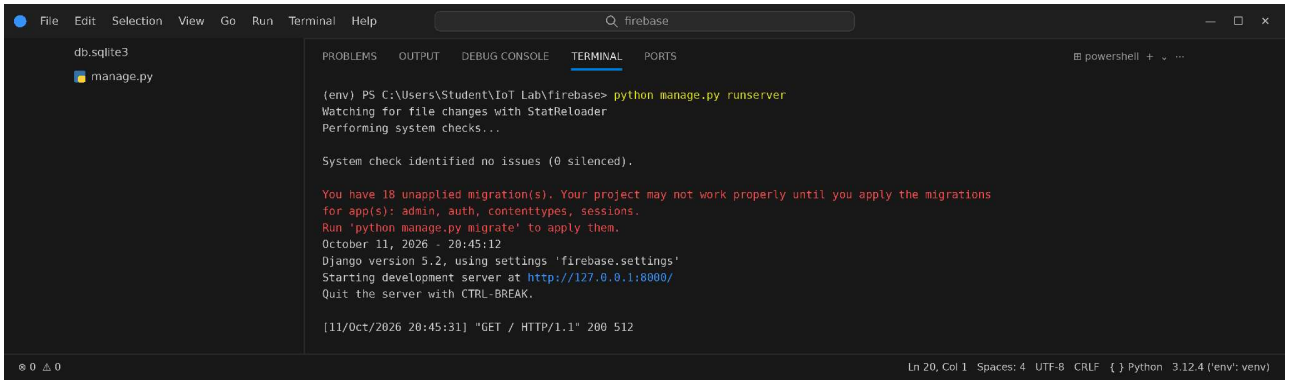


Figure 1: Django server running in VS Code terminal

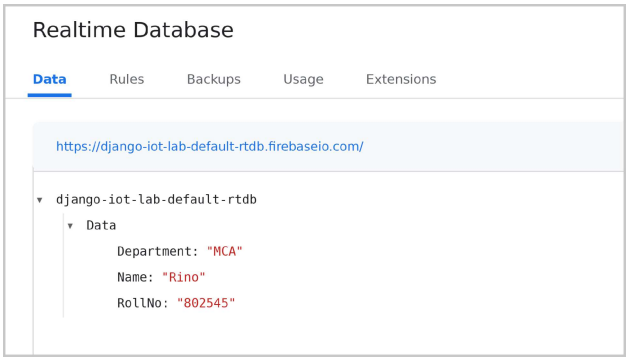


Figure 2: Data stored in Firebase

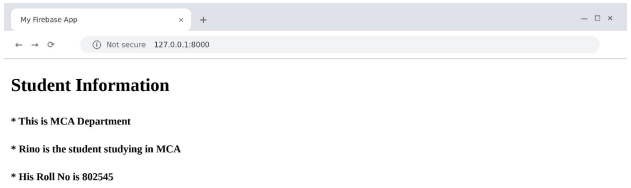


Figure 3: Data shown in browser